

Implementación de RNNs/GRUs para modelar dependencias temporales en datos secuenciales

1. Descripción del Problema

Se implementó una red neuronal recurrente basada en GRU (Gated Recurrent Unit) utilizando PyTorch para modelar dependencias temporales en datos secuenciales asociados a la demanda horaria de bicicletas. El objetivo del modelo es predecir la demanda futura a partir de observaciones históricas multivariadas, aprovechando la capacidad de las arquitecturas recurrentes para capturar patrones temporales y relaciones secuenciales de corto y largo plazo. Los datos fueron organizados mediante ventanas temporales deslizantes de 24 horas (`sequence_length = 24`), donde cada secuencia contiene información histórica utilizada para predecir el valor de la hora siguiente (`delay = 1`). Las variables numéricas fueron normalizadas mediante `StandardScaler` para estabilizar el proceso de entrenamiento.

2. Arquitectura de la Red Neuronal

La arquitectura implementada corresponde a una GRU unidireccional compuesta por una capa recurrente y una capa densa final para regresión. La red recibe secuencias multivariadas con dimensión $(24, n_features)$ y procesa cada timestep de manera secuencial. El último estado oculto generado por la GRU resume la información temporal relevante y es utilizado para realizar la predicción final.

Se incorpora Dropout ($p = 0.2$) como mecanismo de regularización para reducir el riesgo de sobreajuste durante el entrenamiento.

Capa	Tipo	Entrada	Salida	Activación
Entrada secuencial	—	$(24, n_features)$	—	—
Recurrente	GRU	$n_features$	64	$\tanh$ (interna)
Regularización	Dropout	64	64	$p = 0.2$
Salida	Linear	64	1	Lineal

La arquitectura utiliza aproximadamente una capa recurrente (`num_layers = 1`) con 64 unidades ocultas (`hidden_size = 64`), permitiendo modelar relaciones temporales sin incrementar excesivamente la complejidad computacional.

3. Pipeline de Procesamiento

Pipeline aplicado a las particiones de entrenamiento, validación y prueba:

- Ingeniería de características: generación de variables temporales y estadísticas móviles.
- Normalización: aplicación de `StandardScaler` sobre las variables numéricas.
- Construcción de secuencias: ventanas deslizantes de longitud 24.
- Conversión a tensores: adaptación de los datos al formato requerido por PyTorch.
- `DataLoader`: procesamiento por lotes y aleatorización de muestras.

El conjunto de entrenamiento utiliza `shuffle=True` para mejorar la generalización durante el aprendizaje.

4. Parámetros de Entrenamiento

Parámetro	Valor	Justificación
Optimizador	Adam	Convergencia eficiente en redes profundas

Parámetro	Valor	Justificación
Learning rate	0.001	Estabilidad del entrenamiento
Loss function	MSELoss	Adecuada para regresión
Métrica principal	MAE	Interpretación directa del error
Épocas máximas	150	Permite convergencia gradual
Batch size	256	Balance entre memoria y estabilidad
Hidden size	64	Capacidad moderada de representación
Num layers	1	Menor complejidad y sobreajuste
Dropout	0.2	Regularización
Gradient clipping	1.0	Evita explosión de gradientes
Scheduler	ReduceLROnPlateau	Ajuste dinámico del learning rate
Early stopping	Paciencia = 20	Prevención de sobreentrenamiento

Durante el entrenamiento se almacenó automáticamente el mejor modelo según la pérdida de validación. Además, se aplicó reducción adaptativa del learning rate cuando el desempeño dejó de mejorar.

5. Evaluación y Resultados

El modelo fue evaluado utilizando conjuntos de validación y prueba independientes. La métrica principal utilizada fue el MAE (Mean Absolute Error) calculado sobre la escala original de demanda horaria.

Adicionalmente, el desempeño de la red GRU fue comparado contra modelos baseline, incluyendo un enfoque naïve y un modelo HistGradientBoostingRegressor. Los resultados evidenciaron que la arquitectura recurrente logró capturar de manera más efectiva los patrones temporales presentes en la serie, reduciendo el error de predicción respecto a los modelos base.

La implementación demuestra la utilidad de las redes recurrentes tipo GRU para problemas de series temporales, especialmente en escenarios donde existen dependencias secuenciales relevantes entre observaciones consecutivas.